

Introduction

Chapter Contents

Overview	3
Concepts	4
Why Control a System?	4
How Good Is the Hot Plate's Temperature Control?	5
Why Use Electronics and an Embedded Computer?	6
How to Embed a Computer?	7
Examples of Embedded Systems	8
Looking at the Hardware Inside	8
Typical Embedded System Software Operations	12
Embedded System Attributes	12
Interfacing with Inputs and Outputs	13
Concurrency	13
Responsiveness	14
Reliability and Fault Handling	15
Diagnostics	16
Constraints	16
Target Platform	17
Overview	17
Processor	19
Microcontroller	20
Development Board	21
Summary	23
Exercises	24
References	24

Overview

In this chapter we introduce the basic motivations and key concepts for embedding a computer into another system. We examine the design of the hardware and software for the embedded systems and also the constraints that designers face. Finally, we examine the target hardware platform we will use in this textbook.

Concepts

Why Control a System?

Improving how a system is controlled can provide better performance, extra features, reduced purchase or operating costs, and more dependability. An embedded computer system is a computer that has been embedded into a larger system in order to improve it in some way, typically by controlling it.

Figure 1.1 shows an electric hot plate that is used to cook food. The control knob on the front allows the user to turn on or off the hot plate and to set its temperature. Let's examine the existing control system, determine its weaknesses, and consider how we can improve it.

Figure 1.2 shows the internal components of a hot plate: a black control knob, a heating element (covered by a silver metal circle), a red indicator lamp, and a temperature control system. The indicator lamp is connected in such a way that it lights when the heating element is powered.

The temperature control system, shown in Figure 1.3, is very simple and is based on a single component: a temperature-sensitive (thermostatic) switch. One of the switch contacts is made from two different types of metal so that it bends as it grows hotter. When the switch gets hot enough, the contact bends so far that the switch disconnects the heating element from the power. The heating element and then the switch start to cool down. Eventually the switch will cool down enough to bend back and make contact. The control knob adjusts the distance between the switch contacts, setting the temperatures at which the switch will open or close. How well does this control system work? Let's find out.



Figure 1.1 Electric hot plate for cooking food. Photo by author.

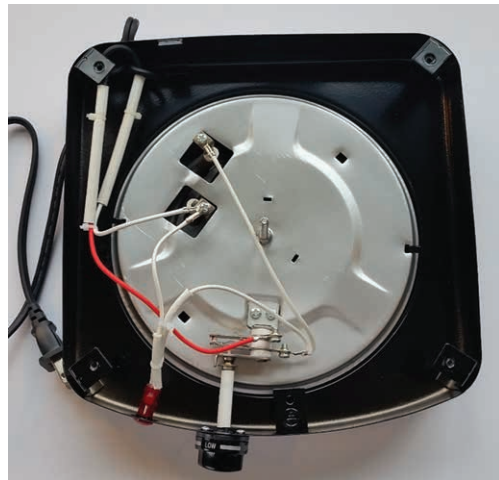


Figure 1.2 Internal components of hot plate. Photo by author.

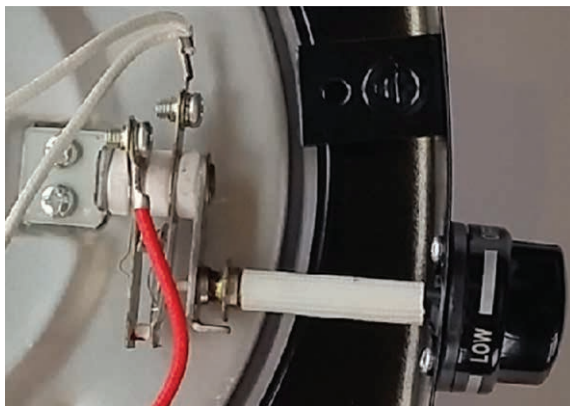


Figure 1.3 A detailed view of the thermostatic switch used to control the heating element. Photo by author.

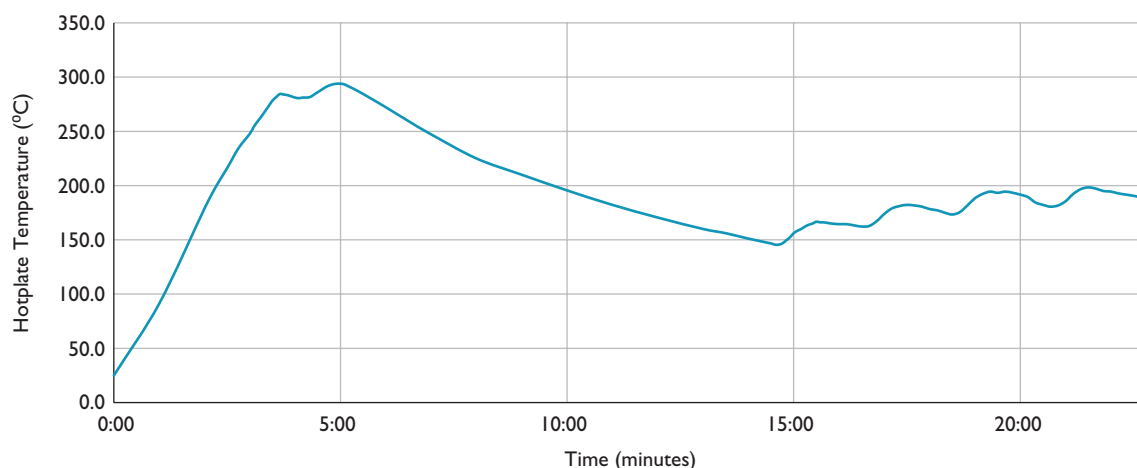


Figure 1.4 Hot plate temperatures measured over time.

How Good Is the Hot Plate's Temperature Control?

The temperature of a hot plate cooking surface can be measured using a thermocouple, a type of electric thermometer. Figure 1.4 shows the temperature over time. At time $t = 0:00$, the control knob is set to Low to turn on the heating element. The temperature rises to nearly 300°C over the next 5 minutes. The heating element turns off briefly at $t = 4:00$ and then for a longer time at $t = 5:00$. The hot plate then takes the next 10 minutes to cool down to about 150°C at about $t = 14:40$. After this start-up period, the temperature starts rising in steps as the switch cycles on and off with a roughly 2-minute period, eventually reaching a range centered near 190°C . Notice the temperature never actually stabilizes, but rises or falls within a range of about 20°C based on whether the switch is open or closed.

Although the control system is simple and low cost, it does not control the temperature of the hot plate very well. Consider the following points:

- The actual temperature swings wildly in the first 15 minutes. Variations in temperature will make it harder to cook food consistently. The temperature finally starts to stabilize after 15 or 20 minutes, which is a long time to wait before starting to cook.
- The actual temperature of the hot plate cooking surface is not measured. Instead, the switch measures a combination of the temperatures of the metal shield and the internal air.
- There is a delay between when the metal shield and the internal air temperatures change and when the switch temperature changes, as the heat must be conducted to the switch contact.
- The control system is not calibrated. The control knob is marked with settings off, low, med, and high instead of actual numerical temperatures.

Why Use Electronics and an Embedded Computer?

Using electronics can improve the temperature control of this hot plate in many ways:

- We can reduce the delay between a temperature change and the control system's response by using a smaller sensor that changes temperature quickly.
- We can mount the smaller temperature sensor on the bottom of the hot plate itself to further reduce the delay. This would save time and power by eliminating the large temperature overshoot at 5 minutes.
- We can switch the heating element on and off more frequently, reducing the temperature ripples shown starting at 15 minutes in the graph.
- If we do this switching fast enough, we can control the fraction of time the heating element is on, giving us a wide range of heat outputs rather than the simple on/off of the current system.
- An electronic approach allows us to measure the temperature precisely instead of relying on the simple above/below information that the thermostatic switch provides.
- Coupling the precise temperature measurement with the proportional heating control, we can use a better control scheme. For example, we can turn on the heater a little bit if the temperature is only slightly below the desired set point, but turn it on more fully if the temperature is far below the set point. This will improve response and reduce the temperature overshoot.
- Adding multiple temperature sensors would allow us to monitor temperatures at multiple locations on the hot plate, not just one. This will further help control the temperature.

We could design a dedicated electronic control circuit to apply these methods, but it is almost always more practical to use an embedded computer because it provides greater flexibility with low cost and a quick development time. The exceptions are devices with extreme requirements (e.g. processing speed, power consumption, or unit cost).

Embedded computers use specialized **integrated circuits (ICs)** called microcontrollers that have features to simplify the monitoring and control of a system. A **microcontroller unit (MCU)** has a **central processing unit (CPU)** that runs a program made of **instructions**.

integrated circuit (IC)

Electronic circuit with components built into a single piece of silicon, enabling extreme miniaturization, mass production, and cost reduction

microcontroller unit (MCU)

Integrated circuit containing CPU, peripherals, support circuits, and often memory

central processing unit (CPU)

Hardware circuit that executes a program's instructions

instruction

Command for processor to execute. Consists of an operation and zero or more operands.

An MCU makes it easier to add sophisticated control methods at a low cost per product. We customize the computer to our application by writing software for the application and designing simple hardware to interface with the system. The recurring hardware costs are low because embedded computers typically use MCUs, which are inexpensive because they are produced in such high volumes. MCUs also reduce system costs because they include circuits to interface with the system, greatly simplifying the hardware development effort and circuit complexity. The main cost for embedded systems is generally the development of the control software, not the hardware itself.

Once there is an MCU in the system, it becomes much easier to add other useful features to improve and differentiate the product:

- Automatically turn off the hot plate for safety after a fixed time with no temperature knob changes.
- Provide calibrated temperature control with actual temperatures on the knob rather than low, med, or hot.
- Flash the lamp to indicate when the hot plate temperature is at or near the set point.
- Display the current temperature and the desired (set-point) temperature on a panel display.

How to Embed a Computer?

Let's examine how to improve the hot plate by adding an embedded computer. Figure 1.5 shows a block diagram of our improved, computer-controlled hot plate. We will use an MCU as the controller to read the desired and actual temperatures and decide on how to control the output. The MCU reads inputs to determine the state of the system being monitored and controlled. For the hot plate, the inputs include the temperature control knob position (desired temperature) and the hot plate temperature. That temperature is measured with a sensor that provides a signal whose voltage varies proportionally with temperature. The MCU may need to convert this signal into a form it can use.

We will use a simple on/off signal to control the heating element. Because the output signals from the microcontroller are low voltage and low current they are not capable of powering the heating element. We need to use a driver circuit to step up the signal from the microcontroller to an adequate level.

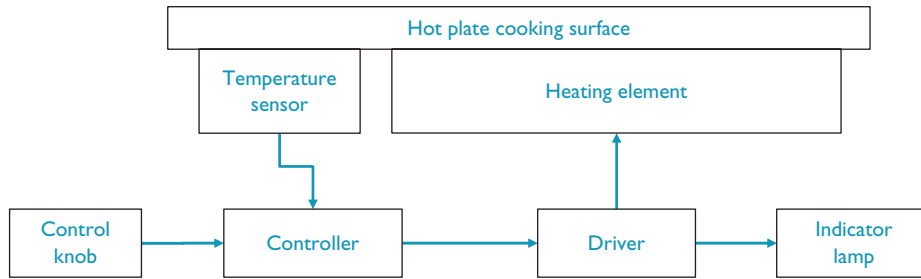


Figure 1.5 Block diagram of a computer-controlled hot plate.

The heating element and the indicator lamp operate at a line voltage, such as 110 V or 220 V. However, the microcontroller operates at a much lower level: 3.3 V. We need to use a power supply to convert the input voltage down to 3.3 V. This power supply is not shown in the diagram.

The main job of the software in the controller will be to compute the error between the desired temperature (the set point), as indicated by the control knob, and the actual temperature, as measured by the temperature sensor. The controller will then adjust the amount of power delivered to the heating element in order to reduce that error.

Examples of Embedded Systems

Let's examine two more examples of embedded systems to get an idea of the variety of devices and their different design goals and constraints.

Figure 1.6 shows a quadcopter, which is a small, remote-controlled toy aircraft. The user flies the quadcopter using a wireless remote control that sends commands to change altitude; rotate; travel forward, back, or sideways; or even flip over 180° and fly upside down. The quadcopter has four motors that drive four rotors to provide lift. By adjusting the speed of the motors individually, the quadcopter can be made to move in different ways. However, it is too difficult for the user to control each motor's speed directly. Instead, the quadcopter's embedded computer translates the user's remote control commands into motor speed commands. This translation depends on the current orientation and motion of the quadcopter, which can change very quickly. The quadcopter has a set of accelerometer sensors to detect acceleration in three directions (up/down, left/right, forward/back) and a set of gyroscopic sensors to detect rotation in three directions (roll, pitch, yaw).

Figure 1.7 shows a refrigerator with a freezer, an ice maker, and a water chiller. Figure 1.8 shows its control panel and display and part of the chilled water and ice dispenser. The refrigerator needs to maintain temperatures within specified ranges in two compartments, allow the user to change those temperatures, light the compartments when its door is opened, make ice, and dispense chilled water, ice cubes, or crushed ice. Other features include sounding a chime if the door is left open for too long and indicating when the water filter needs to be replaced.

Looking at the Hardware Inside

Let's take a look inside the quadcopter. The **printed circuit board (PCB)**, shown in Figure 1.9 and Figure 1.10 is about 4.5 cm long. The two large black squares in Figure 1.10 are the MCU (above) and the acceleration and rotation sensors (below). The corners of the PCB hold four



Figure 1.6 A remote-controlled quadcopter toy. Photo by author.



Figure 1.7 A refrigerator with a freezer, an ice maker, and a dispenser for water and ice (not shown). Photo by author.



Figure 1.8 A user control panel above the chilled water and ice dispenser. Photo by author.

light-emitting diodes (LEDs) and wires to the four motors. A radio interface chip is hidden underneath the white glue and silver cylinder to the right of the MCU and sensors. Most of the small black rectangles with three legs are transistors that drive the four motors. The battery cable plugs into the large orange connector on the left.

printed circuit board (PCB)

Board which holds electronic components and conductive traces for interconnection

light-emitting diode (LED)

Electronic component which emits light. Used for indicators, backlighting, and general illumination.

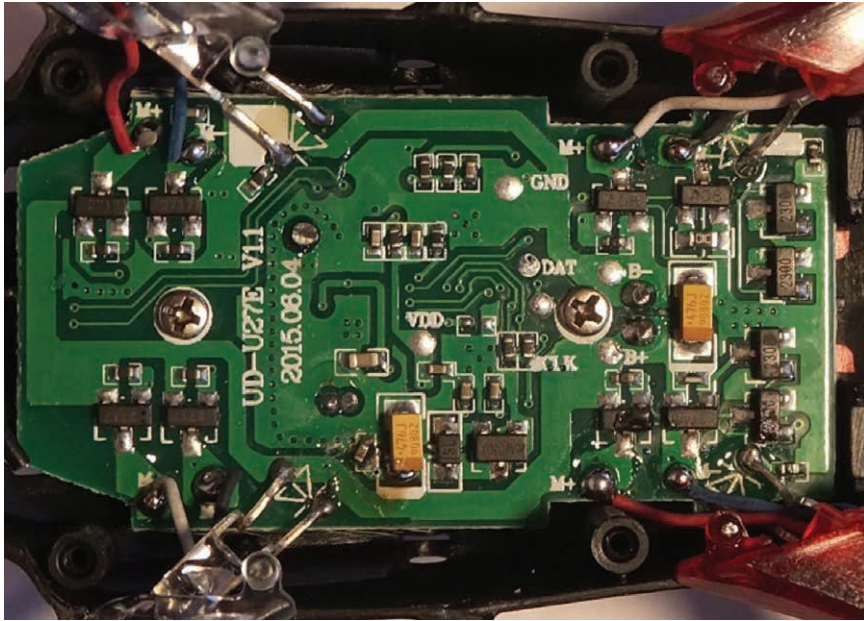


Figure 1.9 The front of a quadcopter controller board. Photo by author.

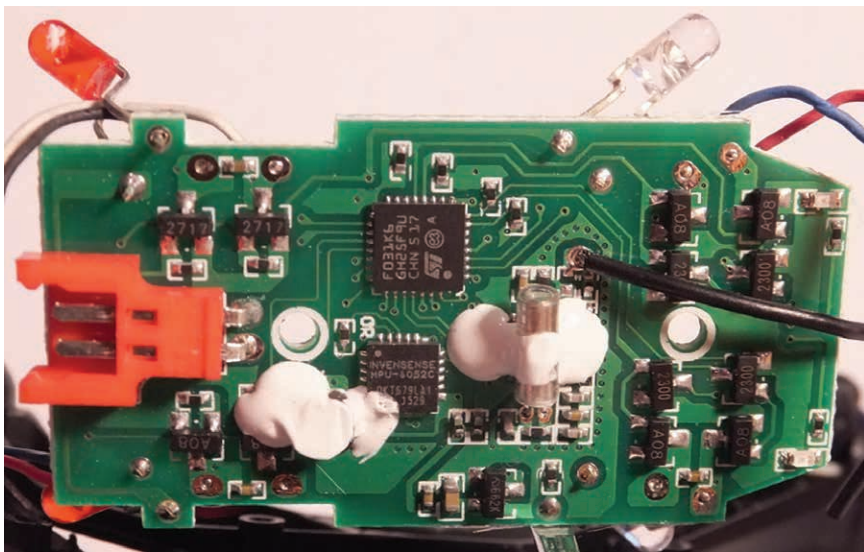


Figure 1.10 The back of a quadcopter controller board. The large black squares are the MCU (top) and the accelerometer/gyroscopic sensor (bottom). The black wire is the radio antenna. Photo by author.

Now let's examine the refrigerator's PCB. Figure 1.11 and Figure 1.12 show the front and back of the main PCB, which is about 25 cm long. The MCU is located on the back of the PCB and is just one of many electronic components. These other components convert power, amplify and condition signals from input devices, and drive output devices. Here are the major input and output devices:

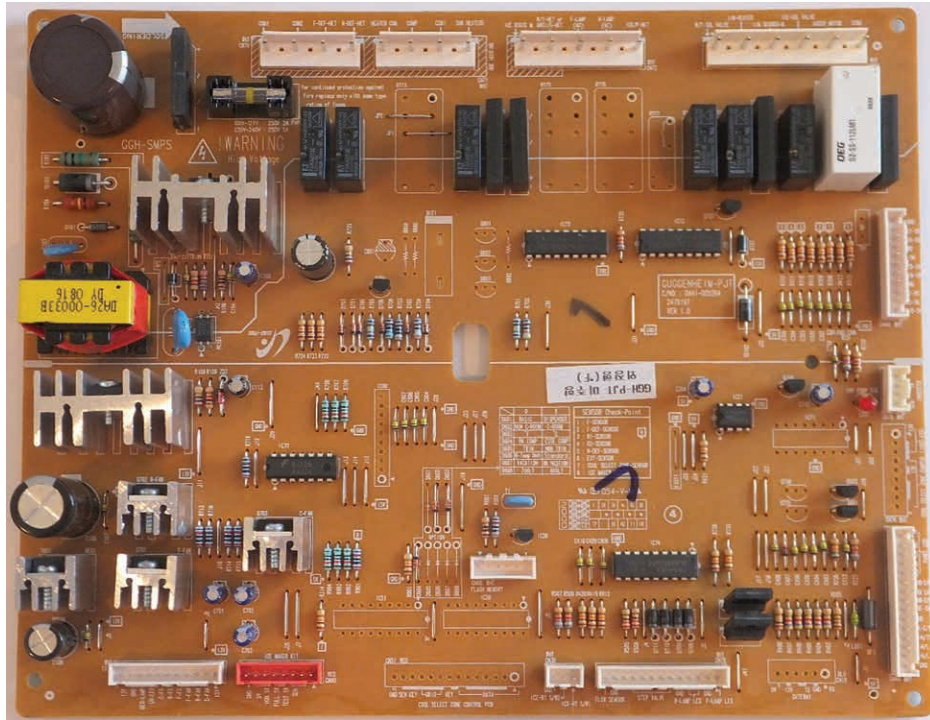


Figure 1.11 The front of a refrigerator controller board. Photo by author.

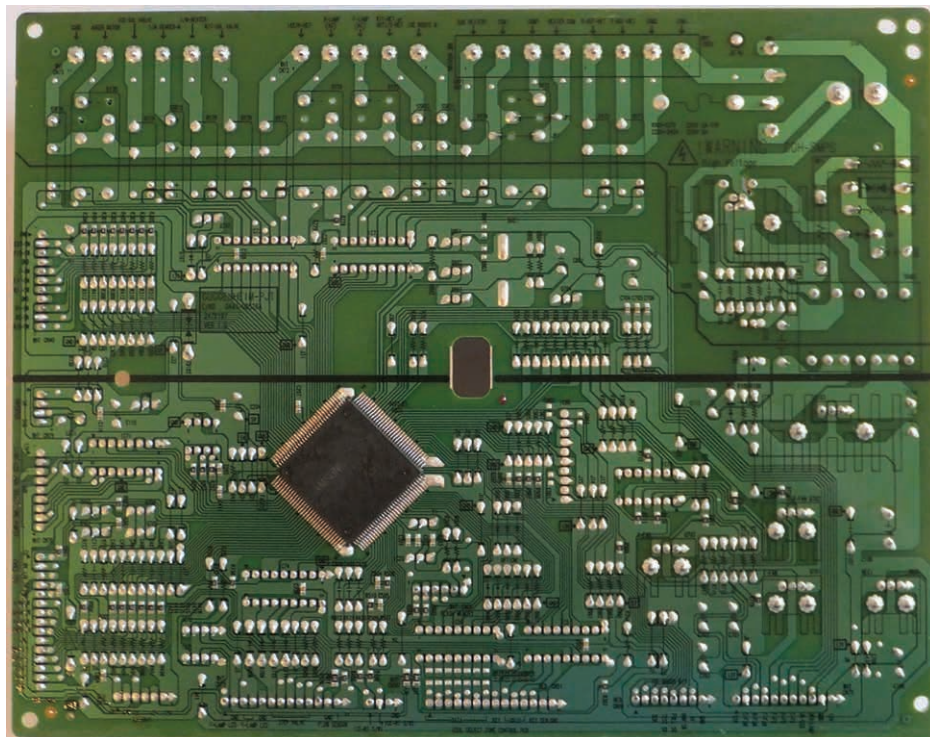


Figure 1.12 The back of a refrigerator controller board. The microcontroller is the large black diamond. Photo by author.

- Inputs
 - Switches: user control panel, compressor overload protection, freezer door, refrigerator door, ice bucket full, water/ice dispenser lever, cube motor position, ice maker test, ice route motor position.
 - Temperature sensors: external air, freezer, freezer defroster, refrigerator, refrigerator defroster.
- Outputs
 - Lights: freezer, refrigerator, dispenser.
 - Indicators: user control panel.
 - Heaters: freezer defroster, refrigerator defroster, water pipe, door cap, dispenser, water tank.
 - Motors: compressor, ice route, auger, cube, ice grinder, freezer fan, condenser fan, refrigerator fan.
 - Water valves: water solenoid, icemaker solenoid, water stepper motor.

You probably didn't expect a refrigerator to be so complex.

Typical Embedded System Software Operations

To do their work, embedded systems typically perform one or more types of operations:

Closed-loop control involves controlling an output variable based on one or more input measurements. For example, the refrigerator controller maintains the temperature by turning on the compressor if the refrigerator compartment is too warm and turning it off if it is too cold. Similarly, the quadcopter controller keeps the craft flat and stable by adjusting the power to its rotor motors based on measurements of acceleration and rotation. There are more sophisticated control methods that consider the size of the error (the difference between the desired value and the actual measured value), how quickly it is changing, and how long it has persisted.

Sequencing involves controlling an output through a sequence of steps. For example, the ice-maker follows several steps to make ice:

- Fill the ice tray with water.
- Chill the ice tray until the water is frozen.
- Heat the ice tray to allow the ice to separate from the tray.
- Eject the ice from the tray.

Signal conditioning and processing may be used to average together multiple sensor readings or filter out noise from motors or other devices. For example, the quadcopter's four motors vibrate as they run, introducing noise into the acceleration and rotation readings. Taking multiple readings and averaging them can reduce the impact of this noise, improving accuracy.

Communications and networking allow the device to interact with subsystems or other systems. The quadcopter receives packets of control data sent by radio from the controller. The MCU needs to decode each packet of data to determine which commands and parameters have been sent.

Embedded System Attributes

Now let's examine some attributes of embedded systems and the impact they have on the software and hardware. These attributes lead to different design approaches and decisions than for personal computer application programs.

Interfacing with Inputs and Outputs

Embedded computers typically need to sense the environment and then control devices in response. To do this, specialized circuits are needed to get the information to and from the CPU, which executes instructions. We have seen examples of input and output devices for the refrigerator and the quadcopter. MCUs consist of a CPU surrounded by specialized *peripheral* hardware circuits that perform much of this interfacing. Any remaining interface circuits are added externally on the PCB.

Many of the external devices use *analog* signals in which the voltage (or current) can take on a continuous range of values to convey information. For example, a temperature sensor might indicate its reading by setting its output signal's voltage to $0.05 \text{ V}/^{\circ}\text{C}$. A reading of 0.5 V would indicate a temperature of 10°C . This analog signal must be converted to a *digital* value for the program to process it; this is done using an *analog-to-digital converter (ADC)*. To generate sounds accurately, the MCU must generate analog voltage signals to drive headphones or speakers. The digital values representing the sound signal can be converted to an analog voltage using a *digital-to-analog converter*.

peripheral

Hardware that helps CPU by interfacing or providing special functionality

analog

Capable of taking on an infinite number of values

digital

Capable of taking on a limited number of values

analog-to-digital converter (ADC)

Circuit which converts an analog value (e.g. voltage) to its corresponding digital value

digital-to-analog converter (DAC)

Circuit which converts a digital value to its corresponding analog value (e.g. voltage)

Often signals must be processed before they are converted to the digital domain for the CPU. For example, weak signals need to be amplified, high-voltage signals need to be scaled down to safe levels, and noise must be filtered out. Similarly, the MCU may not be capable of driving power-hungry output devices (e.g. motors and solenoids), so amplification is needed. Consider again the refrigerator controller's main PCB, as shown in Figures 1.11 and 1.12. The MCU is not powerful enough to drive the heaters, motors, and valves directly, and so it uses various devices (e.g. the black blocks and the white block along the top of the PCB) to do the job. This also protects the circuit by isolating low-voltage components like the MCU (which operates at 3.3 V) from the mains voltage (e.g. 120 V or 220 V).

Concurrency

Embedded controllers must typically manage multiple activities concurrently, often with precise control of the timing:

- For example, the quadcopter MCU must accept user commands by radio while also monitoring rotation (about three axes) and acceleration (in three axes), controlling the speeds of four motors to maintain stable flight, flashing indicator lights, and monitoring battery voltage.
- The refrigerator must control the temperature in the refrigerator compartment, the temperature in the freezer compartment, manage the ice-making process, display information on the front panel, accept user commands, and perform diagnostics.

Adding more features to a system increases the software's complexity. A graphical liquid crystal display (LCD) may require several software components to (1) manage the user interface's windows, menus, and screens, (2) translate text, graphics, and images into pixel values, and (3) update the display's memory with these pixel values. Other examples of such features are WiFi and Bluetooth communication and removable storage devices such as Secure Digital flash cards and USB drives. The maker of a complex peripheral IC may provide a software module (driver) to make interfacing easier. Embedded system developers frequently use third-party software components for such tasks to simplify the development process.

Microcontroller units provide concurrency by sharing the CPU among different parts of the software (including interrupt handlers, tasks, threads, and processes), and also by performing some processing in hardware peripherals that run independently of the CPU.

The scheduler determines what piece of software to run next on the processor and switches execution contexts as needed to make it happen correctly. There is a wide range of schedulers available. The most basic schedulers are simple to use and impose little overhead on the processor. They work best for simple systems and only provide limited help to the developer. More advanced schedulers provide better responsiveness (described next) and more features to help in program development. Real-time operating systems (e.g. RTX from Keil) typically use such schedulers. However, they require more of the processor's time and memory, ruling out the use of some low-performance MCUs. In addition, the presence of additional features introduces some performance variability. A full-fledged operating system (e.g. Linux) provides a wide range of features and services to help the embedded system developer. Such an operating system (OS) requires even more processor time and memory, so a much more powerful MCU is required.

Responsiveness

One challenge for embedded system developers is providing enough responsiveness while sharing the CPU's time across many activities. The field of real-time systems studies this aspect of design and performance.

If an embedded controller does not respond to commands and changes in the environment quickly enough, the system may damage itself, people, or other equipment. Often, embedded systems have requirements that specify deadlines for a response to a given input command. Consider the quadcopter: if there is a one-second delay between your changing the flight controls and the quadcopter acting on that command, the aircraft will be much harder to control.

There are two aspects to making a system responsive: raw processing speed and task scheduling.

First, the processor must be **fast enough** to complete the critical processing before its deadline. There are different ways to approach this. One is to make the code efficient so it can do the

necessary work with very few instructions. Making the code efficient is called **optimization** and is a common task in embedded system development. Another approach is to use a fast processor so that those instructions are executed very quickly. And of course, both can be used.

There are trade-offs involved here: it takes developers more time to optimize code, increasing the development costs but not the recurring per-unit costs. Using a faster processor (usually) does not increase the development costs, but it does increase the recurring per-unit costs. A faster processor is more expensive, and it may also require more careful circuit design methods. Once the processor speed exceeds 50 MHz, the flash memory will not be able to keep up, forcing CPU designers to modify the memory system to hide this delay.

Second, the processor needs to **stay focused** on the **critical processing** rather than being distracted by other processing, which will introduce delays. If possible, the critical processing should be performed by hardware peripherals to prevent any software interference. Otherwise, some kind of software scheduling approach is needed. Simple scheduling approaches provide moderate responsiveness, but to get the best responsiveness, a system needs to use interrupts and a preemptive scheduler (e.g. a real-time kernel or a real-time OS). Stepping up to an OS (e.g. Linux) reduces the system's responsiveness because now there are many activities that could delay the critical processing. Scheduling methods are discussed in Chapter 3.

Some embedded systems have such high processing demands that they use multicore processors, which are able to execute instructions from multiple programs (or parts of the same program) simultaneously. Other such systems may use multiple single-core processors to get higher performance.

Finally, some embedded systems use multiple processors to simplify design, rather than to get raw processing speed. Separating the processing reduces timing interference among the software components. Many commercial off-the-shelf modules (e.g. a WiFi interface) contain an integrated MCU to do the work.

Reliability and Fault Handling

Embedded systems are expected to work correctly and reliably. Unlike personal computers, the user does not expect to have to reboot an embedded computer. If something fails, the system should minimize the impact of that failure rather than cause further problems.

Providing reliability and appropriate fault handling are very much dependent on the specific application. Which components are most likely to fail? There may be methods to detect actual or impending failures and shut down the system before there is more trouble. For example, sensors can be added (e.g. a thermal sensor to detect an overheating motor, a current sensor to detect short-circuited output), the circuit can be designed in a way to make failures easy to detect in software, or the software may be able to analyze historical data to determine failures. Embedded systems software typically contains large amounts of fault-handling code to deal with these exceptional cases.

The quadcopter does not need a long-term, highly reliable operation because it is a toy and the expected lifetime is short (perhaps 6 months). The mechanical components of the quadcopter will likely fail before software faults occur. However, note the white glue in Figure 1.10 which keeps larger, loose objects from vibrating and weakening their connections to the PCB.

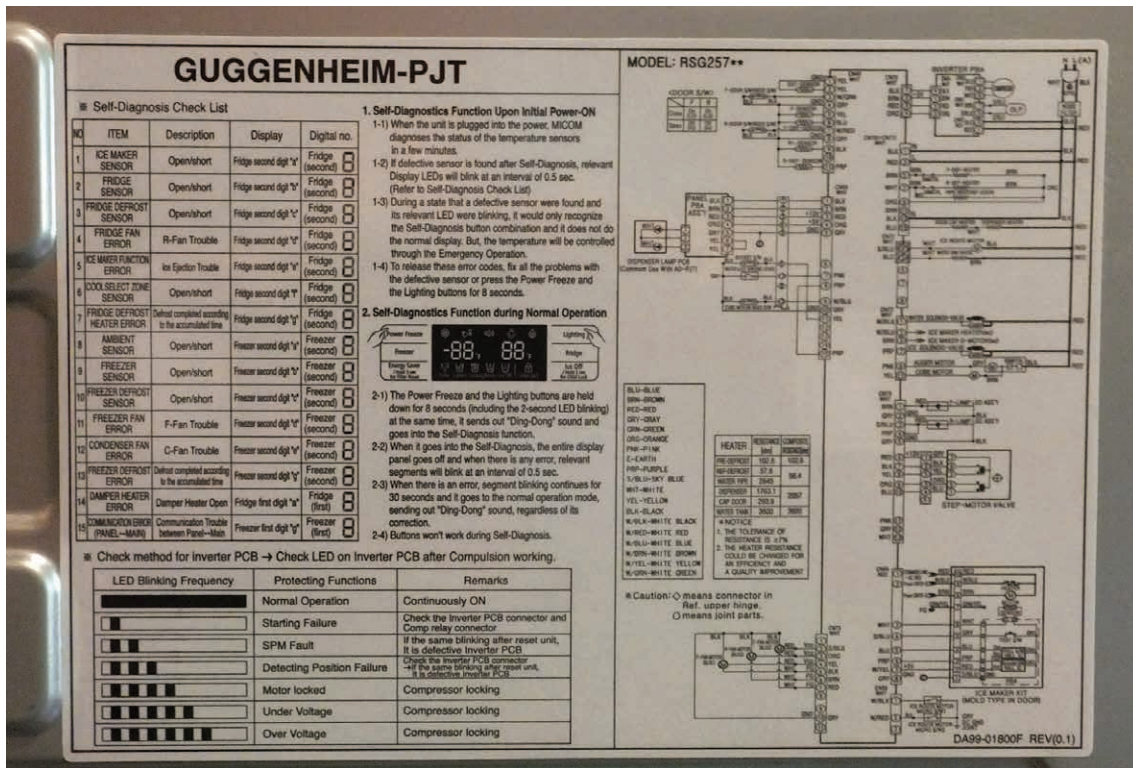


Figure 1.13 The label on the back of a refrigerator provides diagnostic and connection information for service personnel. Photo by author.

Diagnosics

For some systems, it is very important to be able to find and repair faulty components quickly and easily. One good example is the refrigerator: the manufacturer wants to minimize costs and time for its service personnel. The embedded computer system can help by providing diagnostic support to test the system and identify which components have failed.

The left half of the label shown in Figure 1.13 is dedicated to explaining the refrigerator's self-diagnosis capabilities. This allows the manufacturer's service personnel to spend less time identifying the problem, reducing service call costs. The PCB in Figure 1.11 is designed to simplify diagnosis and repair; most connections are labelled, and a legend labelled "SENSOR Check-Point" explains where to measure the eight different temperature sensors.

Constraints

Constraints placed on the resulting embedded system limit the designer's options when trying to meet the system's functional requirements. The embedded system must not be too expensive, large, heavy, power-hungry, and so forth.

Parts **costs** are important for many embedded systems. The quadcopter and its controller sell for about \$30, so the cost for parts (electronic, mechanical, etc.) needs to be under about \$8. There is great pressure to use the least expensive parts that are adequate for the job. In this case, the MCU used is an STM32F031K6, which contains a Cortex-M0 CPU running up to 48 MHz, with 4 kB of SRAM and 32 kB of flash ROM. Most embedded systems are programmed in the C or C++ languages, in large part because these languages can be compiled into code with precise control of the hardware, and yet use small amounts of memory. Similarly, most embedded systems do not use a complex OS such as Linux to share the computer's resources among the parts of the program. The main reasons are that the Linux OS requires large amounts of memory and a fast processor for good performance. There are alternative methods that can use a much less expensive embedded computer and still meet the performance requirements.

Some embedded systems have constraints on **power**. Power constraints limit the system's rate of energy use. For example, the amount of power that a photovoltaic (PV) cell can generate depends on the ambient light. If the circuit tries to use more power than the PV cell can provide, the cell's voltage drops and the system stops working.

Energy constraints limit the total amount of energy that can be used. For example, a battery holds a limited amount of energy. That energy can be used quickly or slowly, but there is only a limited amount available. Flying the quadcopter faster will discharge its battery faster, reducing the maximum flight time possible. So one goal for its developers was to use energy-efficient motors, lights, radios, and MCUs.

Some applications are **weight**-sensitive. Heavier objects take more energy to lift, move, and stop. For the quadcopter, lightness is important because a heavier quadcopter uses energy faster. In order to lift a heavier quadcopter, more lift is needed, so the motors will need to spin faster, drawing more power, and discharging the battery sooner. The quadcopter uses very small components in order to reduce the weight.

The quadcopter also has **size** constraints. Notice how small the components on the PCB are and how closely packed together they are. Compare this with the much larger and more sparsely packed PCB for the refrigerator. The refrigerator's MCU is about 35 mm × 35 mm and takes about 1,225 mm² of the PCB area. This is far larger than the quadcopter's MCU, which at 10 mm × 10 mm takes only 100 mm².

Embedded systems are often expected to operate reliably over a wide range of **temperatures**. Electronic components for consumer applications are expected to operate with ambient temperatures of 0°C to 70°C. Components for the more demanding industrial and automotive applications are expected to handle ranges from -40°C to 85°C.

Target Platform

Overview

Now let's take a look at the hardware platform that we will be using. We will start with the CPU and work our way out. This textbook targets the Arm Cortex-M0 processor shown at the top of Figure 1.14. The processor executes the instructions of the program.

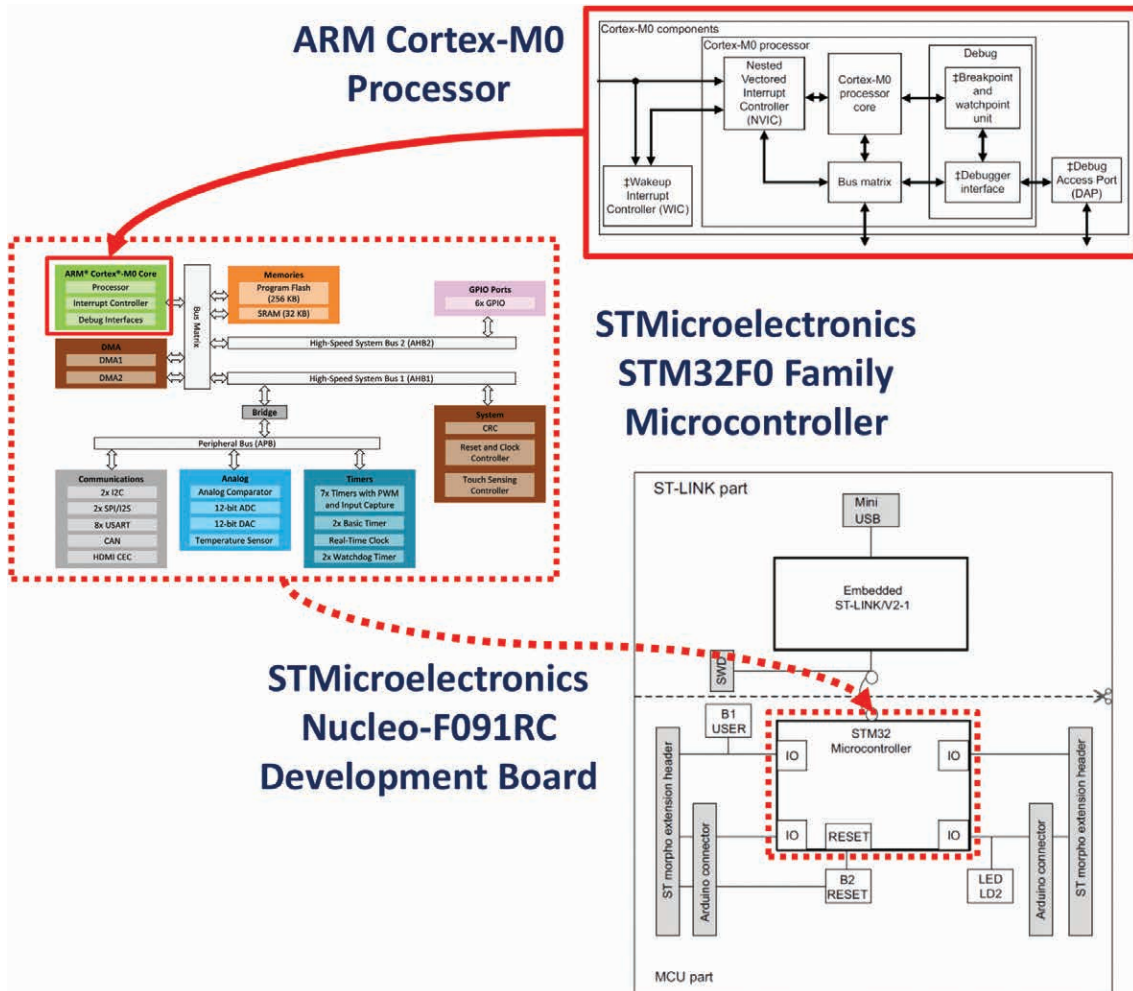


Figure 1.14 An overview of the processor, microcontroller, and development board hardware architectures.

The processor is a component within the microcontroller, shown in the center of Figure 1.14. The MCU adds circuits to clock the processor, a memory to hold the program and data, and peripheral devices that simplify programs and improve their performance. The STM32F091RC MCU is used here.

The microcontroller is mounted on a Nucleo-F091RC development board, shown at the bottom of Figure 1.14 and Figure 1.15. This PCB adds a debug interface, power supplies, and various input and output devices.

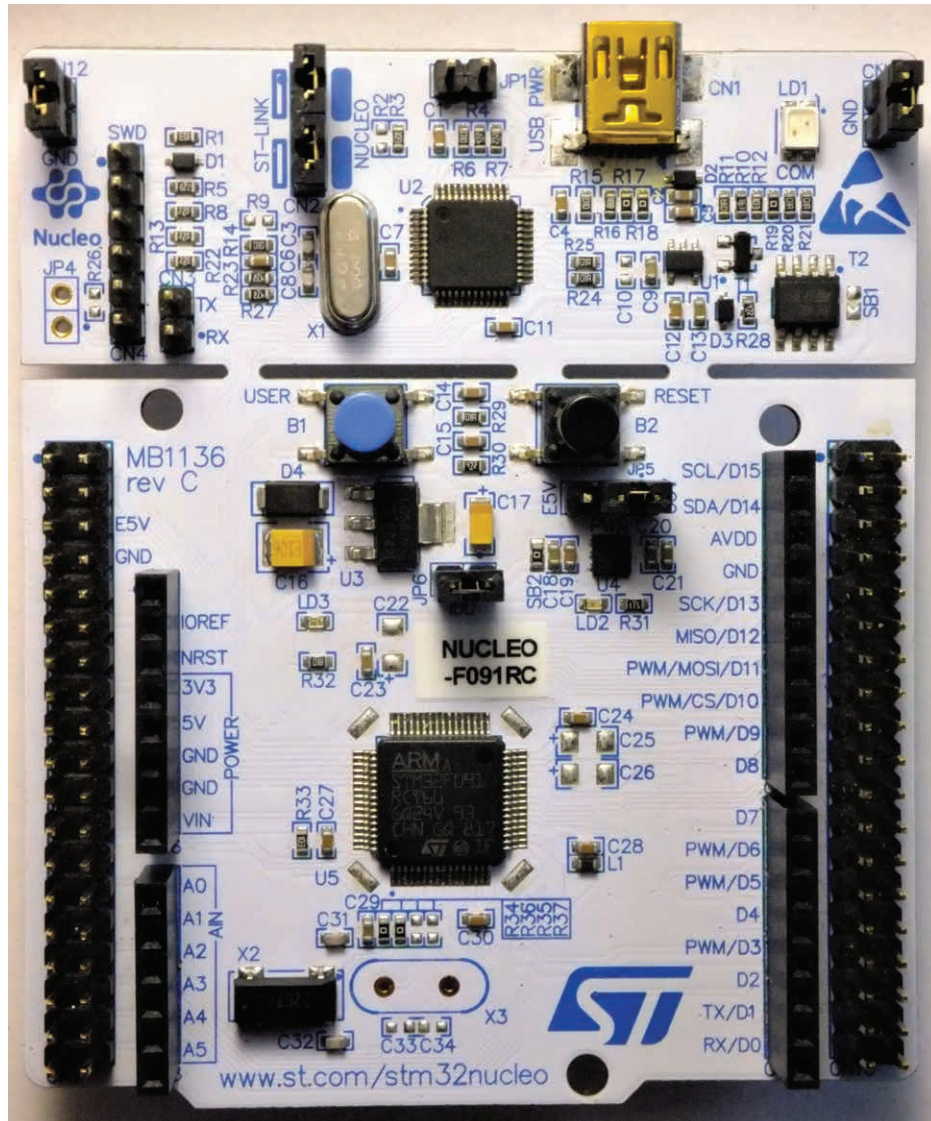


Figure 1.15 Nucleo-F091RC development board from STMicroelectronics. Photo by author.

Processor

The Cortex-M0 processor, shown in Figure 1.16, executes the instructions that make up the program. At the center of the Cortex-M0 processor is the processor core, which communicates with the memory to get the instructions it executes and hold the data it processes. The memory is located outside the processor in the microcontroller and is connected via the AHB-Lite interface. The bus matrix shares the memory bus among multiple possible readers and writers. An optional memory protection unit enables the system to restrict a task to using only a limited region of the memory, limiting the effects of bugs.

Interrupts come to the processor core through the Nested Vectored Interrupt Controller (NVIC), which prioritizes and filters them as needed.

The optional Wakeup Interrupt Controller cuts power consumption by letting the NVIC and the rest of the processor go to a low-power sleep mode and waking them up if an interrupt is requested. Several modules provide debug support, including downloading code to the MCU's memory, setting break points to control program execution, and tracing the sequence of instructions actually executed.

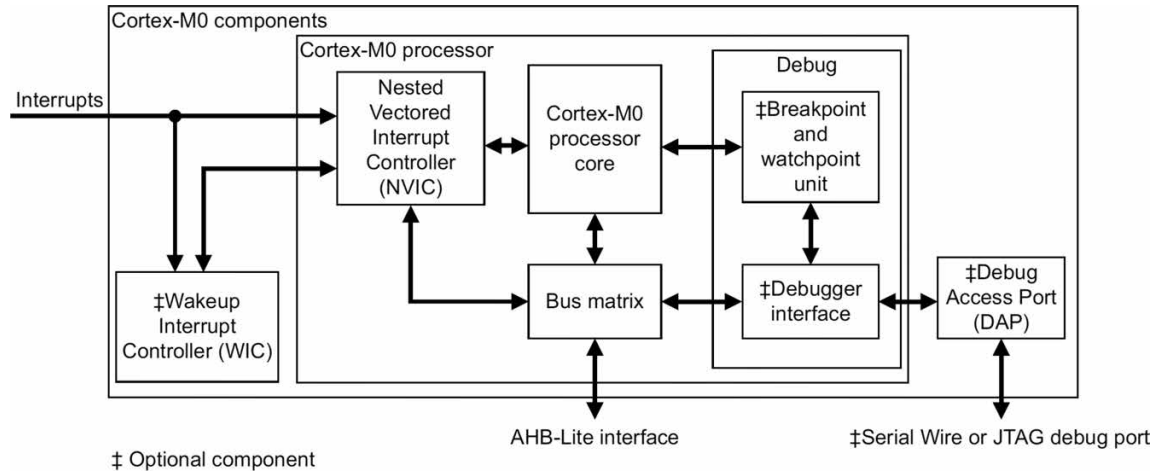


Figure 1.16 Arm Cortex-M0 processor and its associated components. Courtesy of Arm Ltd.

Microcontroller

The microcontroller augments the Arm Cortex-M0 processor by adding memory and supporting circuitry and peripheral devices (Figure 1.17). There are typically two types of memory provided. The flash memory retains its contents even without power, so it holds the program and fixed data. The SRAM does not retain its contents if there is no power and is used for temporary data storage.

The MCU used here (STM32F091RC) features a Cortex-M0 processor capable of running up to 48 MHz and contains 256 kB of flash ROM, 32 kB of RAM, and a wide range of peripherals [1], [2].

The support circuitry is required to make the processor operate. For example, the processor requires a clock signal, and the MCU adds multiple clock generators. These clocks have different levels of accuracy, speed, power consumption, and configurability.

Peripheral devices off-load work from the program (which executes on the processor) and perform it in hardware. For example, **timer** peripherals allow precise time measurement of events or input signals. They also enable generation of repetitive signals without software overhead.

Communication interfaces translate digital data between the processor's format and the formats

which external devices use. **Analog interfaces** translate data between the processor's digital format and the analog domain, in which signals vary across a continuous range of voltages, rather than the binary 1/0 of digital electronics. Finally, other peripherals are used to reset an out-of-control program (watchdog), accelerate and automate memory transfers (DMA), and perform other tasks.

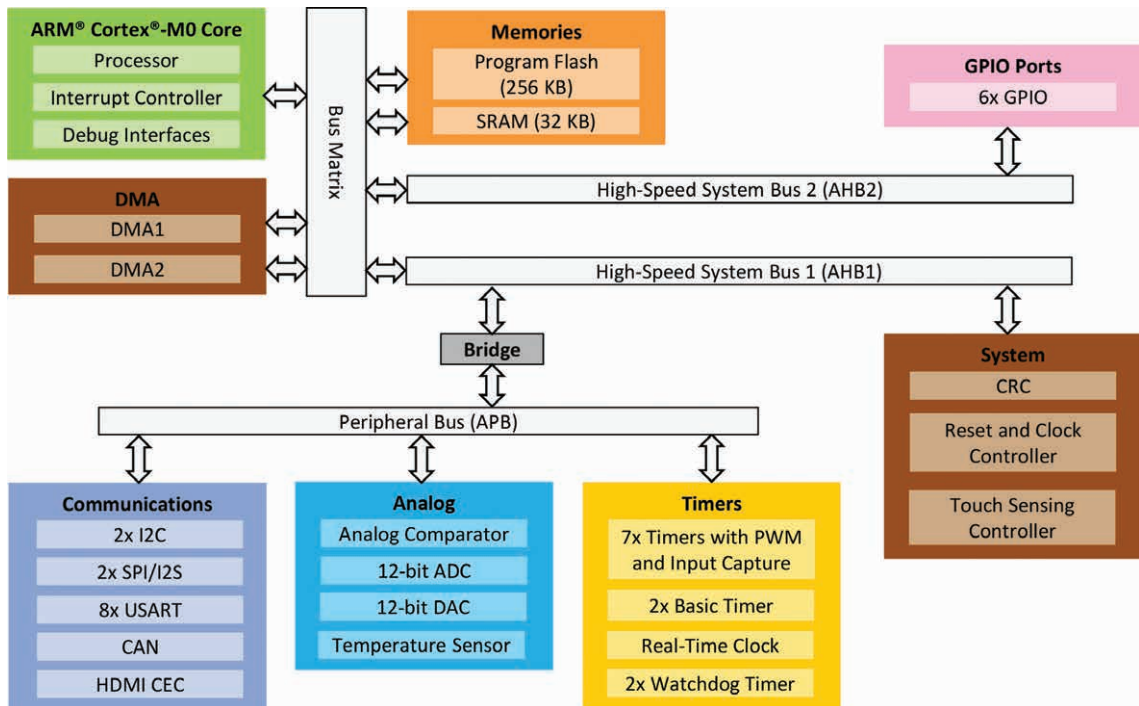


Figure 1.17 STM32F0 family microcontroller (MCU).

Development Board

As shown in Figures 1.15 and 1.18, the microcontroller is mounted on a Nucleo-F091RC development board [3]. This board adds a power supply, a debug interface, and various input and output devices. A 32 MHz clock source provides a stable, accurate timing reference.

The board is normally powered by the 5 V supplied by the USB connection. A voltage regulator drops this to 3.3 V in order to operate the board's components. The power can be provided by an external source, allowing the board to operate without a connection to USB power.

A separate **debug microcontroller** (in the box labeled ST-LINK) translates commands and data between the development PC (via a USB connection) and the STM32F091RC MCU (the **target microcontroller**).

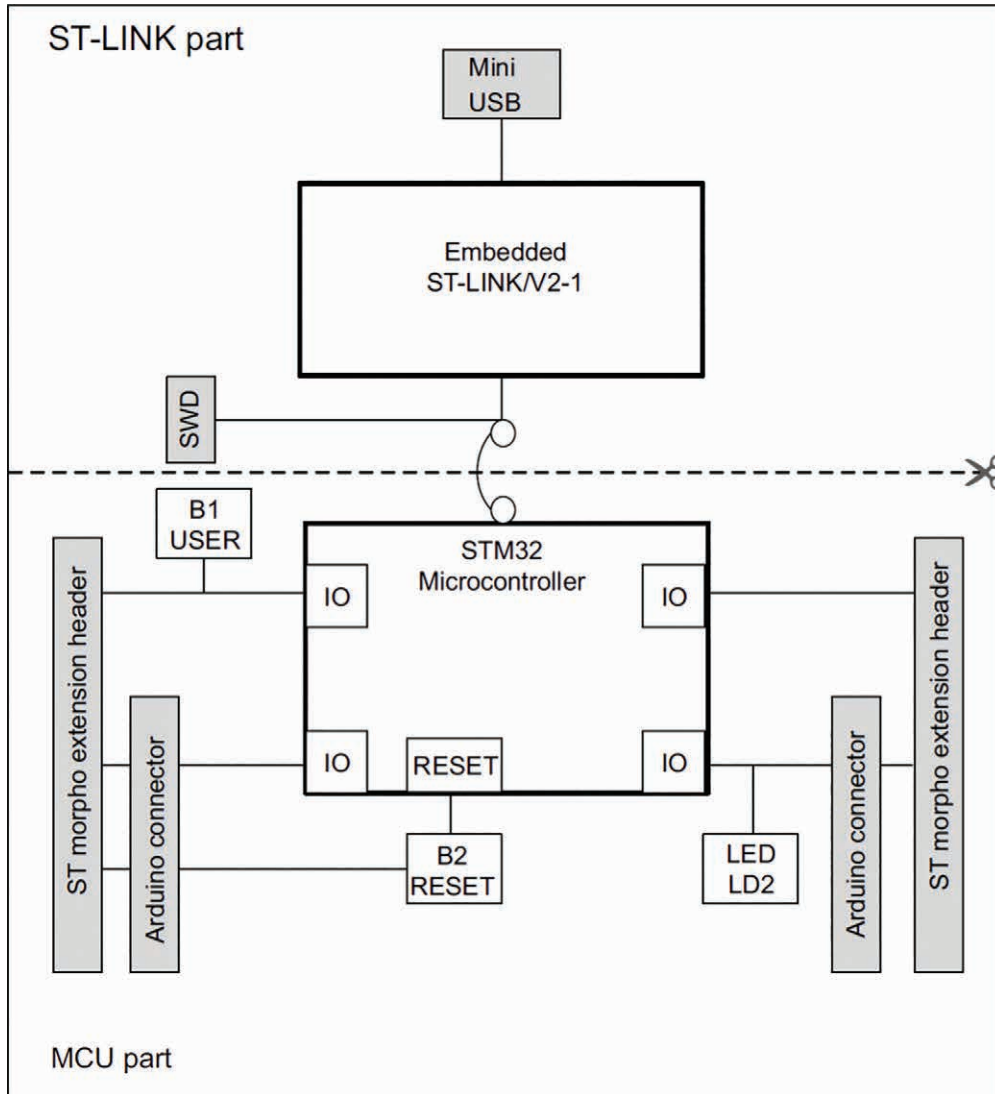


Figure 1.18 Hardware block diagram of Nucleo-F091RC development board. [1]

The X-NUCLEO-IKS01A1 expansion board is used in Chapter 7 and Chapter 8 [4]. It contains motion MEMS and environmental sensors. The expansion board is plugged into the Nucleo-F091RC board as shown in Figure 1.19. We will use the 3-axis accelerometer and 3-axis gyroscope to measure the inclination of the board. It contains also sensors for temperature, humidity and air pressure.

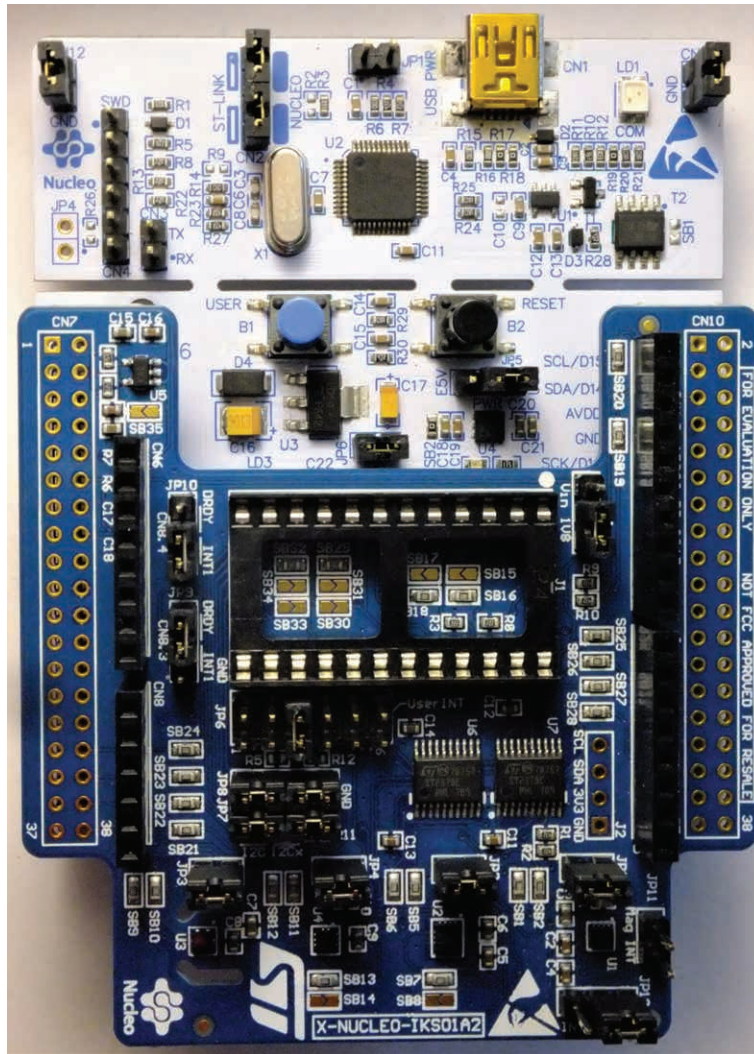


Figure 1.19 X-NUCLEO-IKS01A1 expansion board plugged into the Nucleo-F091RC development board. Photo by author.

Summary

Embedding a computer to control a system can provide benefits such as better performance and sophisticated features. We have seen three examples of applications: a hot plate for cooking food, a toy quadcopter aircraft, and a refrigerator. The embedded computer is typically made from a microcontroller that runs specialized control software which monitors the critical aspects of the system and its environment and then adjusts its outputs so the system behaves as needed. The computer

uses special hardware to interface with the inputs and outputs. To meet application requirements, embedded systems must provide sufficient concurrency, responsiveness, reliability, fault handling, and diagnostic help while meeting constraints on cost, size, weight, power, energy, and temperature. Finally, we have seen the target platform for this textbook: the Arm Cortex-M0 processor core within the STM32F091RC MCU on the STMicroelectronics' Nucleo-F091RC development board.

Exercises

1. You have been asked to add an “automatic power-off” controller to a flashlight. The flashlight should turn off automatically after 3 minutes if it is not being used any more. The controller must be small, inexpensive, and use little power.
 - a. Can you think of a way to do this with electronics but without an embedded computer?
 - b. Can you think of a way to do this with a mechanical approach?
 - c. Now, really thinking outside the box, can you think of a way to do this with a pneumatic approach (using air)?
2. Consider embedding a computer in a flashlight. List three benefits or new useful features that are now possible. How much more would you be willing to pay for each feature?
3. A modern automobile has dozens of embedded computers. However, the auto industry is extremely cost-sensitive, so there must be a compelling reason to add computers. Give five examples of features enabled by embedded computers, and explain what major benefit(s) they provide.
4. Look around and find five devices that are likely to have embedded computer control systems. Answer these questions for each device:
 - a. What does the device do?
 - b. What are the device's inputs?
 - c. What are the device's outputs?
 - d. How does the embedded computer improve the device? Does it provide more features? Does it improve performance?
 - e. What are the biggest constraints on the device (size, cost, power, etc.)?
 - f. Would it be possible to build the device using a different kind of controller (e.g. mechanical)? How would that affect the device's features, performance, cost, size, and weight?

References

- [1] STMicroelectronics NV, *User Manual UM1724: STM32 Nucleo-64 Boards*, 2019.
- [2] STMicroelectronics NV, *Reference Manual RM0091: STM32F0x1/STM32F0x2/STM32F0x8*, 2017.
- [3] STMicroelectronics NV, *STM32F091xB STM32F091xC Data Sheet*, DocID 026284, 2017.
- [4] STMicroelectronics NV, *User Manual UM2121: Getting Started with the X-NUCLEO-IKS01A2 Motion MEMS and Environmental Sensor Expansion Board for STM32 Nucleo*, DocID 029834, rev. 2, 2017.

2